WashU-2 CPU

Week 4 Report

ALU & Memory — Mid-Submission

Author	Ayush Pati
Roll No.	25B3901
Email	25b3901@iitb.ac.in
Institution	IIT Bombay — Electrical Engineering, Second Year
Programme	SOC 2026 — WashU-2 CPU Mentorship
Submitted	Week 4 · June 2026 (Mid-Submission)

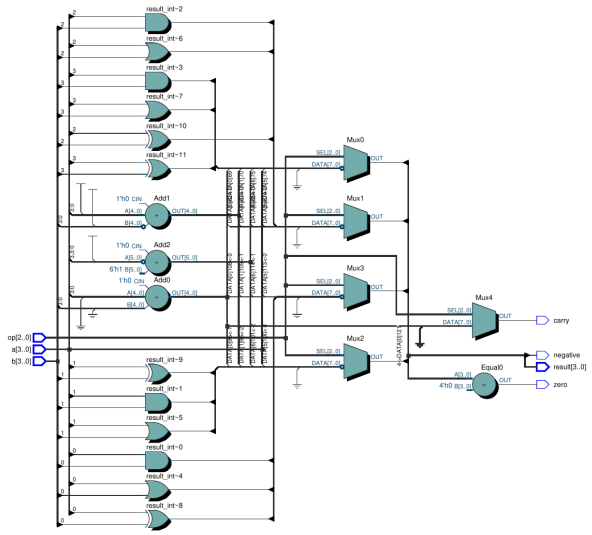
Exercise 1 — 4-bit ALU with Carry, Negative & XOR Flags

Entity: `alu4_extended`

A purely combinational 4-bit ALU supporting 8 operations selected by a 3-bit op code: ADD, SUB, AND, OR, XOR, NOT, NEGATE, and NOP. Three flag outputs accompany the result: **zero** (result = 0), **carry** (unsigned overflow from ADD), and **negative** (MSB of result). Carry detection uses 5-bit widened operands — `'0' & unsigned(a)` zero-extends a to 5 bits; the fifth bit of the sum is the carry out. Internal signals `result_int` and `carry_int` allow flags to be computed from the result without reading output ports. The combinational process uses explicit defaults at the top to prevent unintended latches on undefined op values.

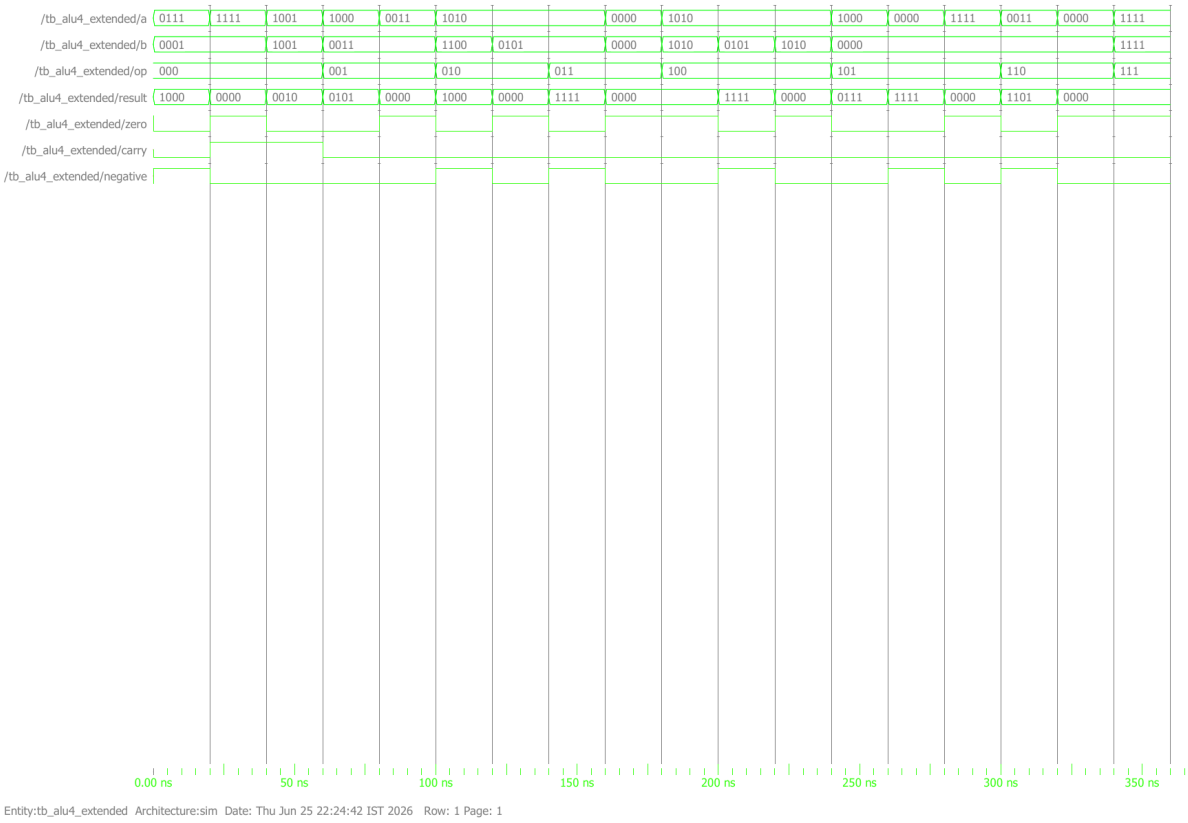
RTL Netlist View

Generated from Quartus RTL Viewer. Shows synthesised gate-level logic before technology mapping.



Simulation Waveform

Captured from ModelSim after running RTL simulation. All input and output signals are visible.


/tb_alu4_extended/a
/tb_alu4_extended/b
/tb_alu4_extended/op
/tb_alu4_extended/result
/tb_alu4_extended/zero
/tb_alu4_extended/carry
/tb_alu4_extended/negative
0111 1111 1001 1000 0011 1010 0000 1010 1000 0000 1111 0011 0000 1111
0001 1001 0011 1100 0101 0000 1010 0101 1010 0000 1111
000 001 010 011 100 101 110 111
1000 0000 0010 0101 0000 1000 0000 1111 0000 1111 0000 0111 1111 0000 1101 0000
0.00 ns 50 ns 100 ns 150 ns 200 ns 250 ns 300 ns 350 ns
Entity:tb_alu4_extended Architecture:sim Date: Thu Jun 25 22:24:42 IST 2026 Row: 1 Page: 1

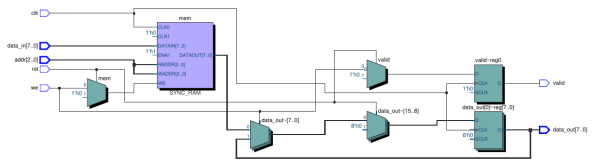
Exercise 2 — 8×8 Synchronous RAM with Read-Valid Handshake

Entity: `ram8x8`

A synchronous single-port RAM with 8 locations of 8 bits each (3-bit address). All operations are clocked on `rising_edge(clk)`. On a write (`we='1'`), data is stored and `valid` is cleared immediately. On a read (`we='0'`), the addressed word is registered into `data_out` and `valid` is asserted — both appear one cycle after the read request, implementing a one-cycle read latency handshake pattern used in pipelined memory interfaces. Synchronous reset clears `data_out` and `valid` but leaves RAM contents intact, consistent with real FPGA block RAM behaviour.

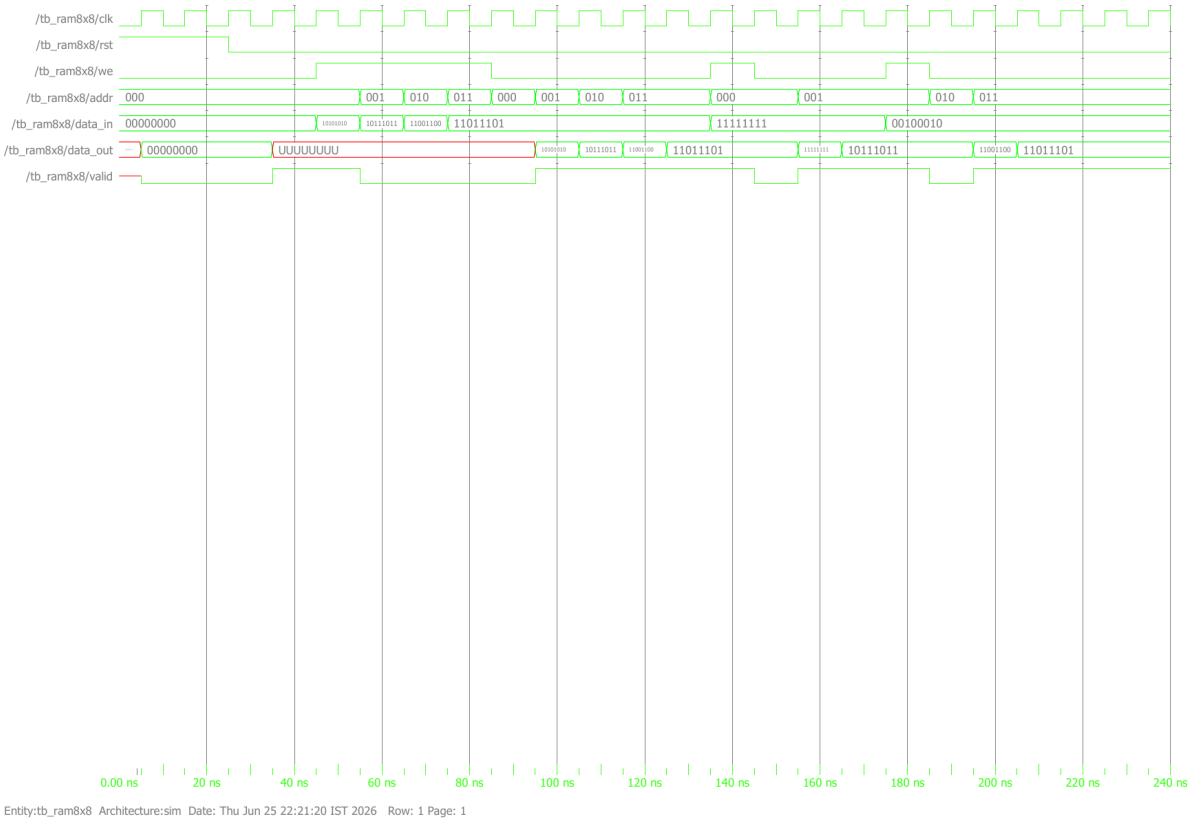
RTL Netlist View

Generated from Quartus RTL Viewer. Shows synthesised gate-level logic before technology mapping.



Simulation Waveform

Captured from ModelSim after running RTL simulation. All input and output signals are visible.


/tb_ram8x8/clk
/tb_ram8x8/rst
/tb_ram8x8/we
/tb_ram8x8/addr
/tb_ram8x8/data_in
/tb_ram8x8/data_out
/tb_ram8x8/valid
000
001
010
011
000
001
010
011
000
001
010
011
00000000
11011101
11111111
00100010
00000000
UUUUUUUU
11011101
10111011
11011101
0.00 ns
20 ns
40 ns
60 ns
80 ns
100 ns
120 ns
140 ns
160 ns
180 ns
200 ns
220 ns
240 ns
Entity:tb_ram8x8 Architecture:sim Date: Thu Jun 25 22:21:20 IST 2026 Row: 1 Page: 1

Bugs & Debugging Notes

The bugs this week span both exercises and reflect the most common pitfalls when working with arithmetic types and synchronous memory in VHDL for the first time.

Bug 1 — Reading an Output Port to Compute Flags (alu4_extended)

The zero and negative flags were initially computed by reading the `result` output port directly inside the process:

```
-- Driving result directly, then trying to read it back:
result <= std_logic_vector(sum5(3 downto 0));
zero <= '1' when result = "0000" else '0'; -- ■ reading output port
negative <= result(3); -- ■ reading output port
```

Fixed:

```
-- Internal signal mirrors the output:
signal result_int : std_logic_vector(3 downto 0);
...
result_int <= std_logic_vector(sum5(3 downto 0)); -- drive internal
result <= result_int; -- connect to port
zero <= '1' when result_int = "0000" else '0'; -- ■ read internal
negative <= result_int(3); -- ■ read internal
```

In VHDL-93, output ports are write-only — reading them back inside the same architecture is illegal and causes a compilation error. The standard fix is to introduce an internal signal (`result_int`) that mirrors the output: all internal logic reads `result_int`, and a single concurrent assignment drives the output port from it. VHDL-2008 relaxes this restriction, but Quartus defaults to VHDL-93 mode, so the internal-signal pattern is the reliable approach.

Bug 2 — Missing Process Sensitivity List Entry Causes Stale Output

The ALU combinational process initially omitted `sum5` from the sensitivity list. Since `sum5` is a signal computed by a concurrent statement outside the process, the process did not re-evaluate when the addition result changed:

```
process(a, b, op) -- ■ sum5 missing from sensitivity list
begin
case op is
when "000" =>
result_int <= std_logic_vector(sum5(3 downto 0)); -- stale sum5!
carry_int <= sum5(4);
...
end case;
end process;
```

Fixed:

```
process(a, b, op, sum5) -- ■ sum5 included
begin
...
end process;
```

A combinational process re-evaluates only when a signal in its sensitivity list changes. If `sum5` is omitted, the process uses its last-computed value even after a or b change the addition result — producing correct output on the first evaluation but stale results on subsequent input changes. The symptom is subtle: the waveform looks correct immediately after reset but shows wrong carry/result values after the first input transition. Every signal read inside a combinational process that is not itself driven by the process must appear in the sensitivity list.

Bug 3 — Reading valid One Cycle Too Early (ram8x8 Testbench)

The testbench checked `valid` immediately after asserting a read — on the same clock edge as the read request — rather than waiting one cycle for the registered output to appear:

```
-- Issue a read:
we <= '0';
addr <= "010";
wait until rising_edge(clk);
-- Check valid immediately: ■
assert valid = '1' report "valid not asserted" severity error;
```

Fixed:

```
-- Issue a read:
we <= '0';
addr <= "010";
wait until rising_edge(clk);
-- Wait ONE more cycle for registered output: ■
wait until rising_edge(clk);
assert valid = '1' report "valid not asserted" severity error;
assert data_out = expected_data report "wrong read data" severity error;
```

The `ram8x8` design is fully synchronous — `data_out` and `valid` are registered outputs that update one clock cycle *after* the read request edge. Checking them on the same edge as the request samples their old values, which are either `'0'` (after reset) or the previous read's data. This is the standard read-latency handshake: the consumer must wait for `valid='1'` before trusting `data_out`. The same one-cycle latency applies to all synchronous RAM designs and is a fundamental property of registered memory outputs in FPGA block RAM.